Python Scripts Documentation

app.py

Description: app.py ———————————————————————————————— Streamlit Web UI for the Arabic Medical Chatbot.

Run: streamlit run app.py

Functions & Classes:

- **Function** `load_chatbot_v3` : Load the crew (v3 — busts old cache).
- **Function** `_extract_final_answer` : Parse crew.run() JSON output: {final_answer, meta}. Returns (answer, citations, is_emergency, meta). Falls back gracefully to raw text.

---

get_categories.py

Functions & Classes:

- No main functions or classes defined.

---

test_eval.py

Functions & Classes:

- **Function** `main` : No description available.

---

test_faiss.py

Functions & Classes:

- **Function** `main` : No description available.

---

src\medical_chatbot\crew.py

Description: crew.py ———————————————————————————————— CrewAI crew definition for the Arabic Medical Chatbot. Crew name: arabic_chatbot

Assembles 6 agents in a sequential process:

1. Language Detection Agent
2. Medical Classification Agent
3. Hybrid Retrieval Agent
4. Citation Grounding Agent
5. Arabic Medical Response Agent
6. Hallucination Detection Agent

Functions & Classes:

- **Function** `_build_llm` : Build LLM client from environment config.
- **Class** `arabic_chatbot` : Arabic Medical Chatbot Crew Multi-agent system for answering medical questions in Arabic using hybrid RAG (FAISS + BM25) and SentenceTransformers embeddings.

- **Method** `__init__` : No description available.
- **Method** `_lang_tool` : No description available.
- **Method** `_cls_tool` : No description available.
- **Method** `_search_tool` : No description available.
- **Method** `_citation_tool` : No description available.
- **Method** `_hallucination_tool` : No description available.
- **Method** `language_detection_agent` : No description available.
- **Method** `medical_classification_agent` : No description available.
- **Method** `hybrid_retrieval_agent` : No description available.
- **Method** `citation_grounding_agent` : No description available.
- **Method** `arabic_medical_response_agent` : No description available.
- **Method** `hallucination_detection_agent` : No description available.
- **Method** `language_detection_task` : No description available.
- **Method** `medical_classification_task` : No description available.
- **Method** `hybrid_retrieval_task` : No description available.
- **Method** `citation_grounding_task` : No description available.
- **Method** `arabic_medical_response_task` : No description available.
- **Method** `hallucination_detection_task` : No description available.
- **Method** `crew` : No description available.
- **Method** `run` : Direct pipeline with retrieval mode and live progress support. mode: 'rag' | 'bm25' | 'internet' | 'hybrid' | 'all' on_progress: optional callable(step, label, detail) for live UI updates

---

src\medical_chatbot\main.py

Description: main.py ——————————————————————————————— CLI entrypoint for the Arabic Medical Chatbot.

Usage: # Interactive mode python src/medical_chatbot/main.py

```
# Single query
python src/medical_chatbot/main.py --query "ما هي أعراض ارتفاع ضغط الدم؟"
```

Functions & Classes:

- **Function** `run_query` : Run the crew pipeline on a single query.
- **Function** `cli` : No description available.

---

src\medical_chatbot_*init*_.py

Functions & Classes:

- No main functions or classes defined.

---

src\medical_chatbot\rag\build_index.py

Description: build_index.py ——————————————————————————————— One-shot CLI script to build and save both indexes:

1. FAISS vector index (dense retrieval)
2. BM25 keyword index (sparse retrieval fallback)

Usage: # Full dataset (may take 15-30 min on CPU) python src/medical_chatbot/rag/build_index.py

```
# Quick test with 1000 rows
python src/medical_chatbot/rag/build_index.py --sample 1000
```

Functions & Classes:

- **Function** `parse_args` : No description available.
- **Function** `main` : No description available.

---

src\medical_chatbot\rag\chunking.py

Description: chunking.py ———————————————————————————————— Splits MedicalDocuments into fixed-size token chunks using tiktoken (cl100k_base tokenizer).

Chunk size : 500 tokens Overlap : 100 tokens

Functions & Classes:

- **Class** `TextChunk` : A text chunk derived from a MedicalDocument.
- **Class** `Chunker` : No description available.
  - **Method** `__init__` : No description available.
  - **Method** `_chunk_tokens` : Split a token list into overlapping windows.
  - **Method** `chunk_documents` : Chunk all documents into overlapping token windows. For short Q&A pairs that fit within chunk_size, a single chunk is produced (no splitting needed).
- **Function** `chunk_documents` : Convenience function wrapping the Chunker class.

---

src\medical_chatbot\rag\document_loader.py

Description: document_loader.py ———————————————————————————— Loads the Arabic medical Q&A CSV dataset.

Dataset columns: q_body – Arabic question text a_body – Arabic answer text category – Arabic category label category_en – English category label

Functions & Classes:

- **Class** `MedicalDocument` : A single medical Q&A document.
- **Function** `load_documents` : Load medical Q&A documents from CSV. Args: csv_path: Path to concatenated_df.csv sample: If set, randomly sample this many rows (for testing). min_answer_len: Drop rows where the answer is too short. Returns: List of MedicalDocument objects.

---

src\medical_chatbot\rag\embedding_pipeline.py

Description: embedding_pipeline.py ———————————————————————————— Generates dense vector embeddings using SentenceTransformers. Default model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

Functions & Classes:

- **Class** `DocumentEmbedder` : Wraps SentenceTransformers for producing sentence embeddings. Usage: embedder = DocumentEmbedder() vecs = embedder.embed(["مرحبا", "كيف حالك"]) # np.ndarray

    - **Method** `__init__` : No description available.
    - **Method** `embedding_dim` : No description available.
    - **Method** `embed` : Embed a list of texts. Returns: numpy array of shape (len(texts), embedding_dim), L2-normalized.
    - **Method** `embed_query` : Embed a single query string. Returns shape (1, dim).
- **Function** `get_embedder` : Return the global embedder (loaded once).

---

src\medical_chatbot\rag\keyword_index.py

Description: keyword_index.py ——————————————————————————————— BM25 keyword index for sparse retrieval fallback.

Uses `rank_bm25` (Okapi BM25) on Arabic tokenized text. The index is saved as a pickle file for fast re-loading.

Functions & Classes:

- **Function** `_tokenize_arabic` : Simple Arabic whitespace + punctuation tokenizer. Removes diacritics and normalizes for BM25 matching.
- **Class** `BM25KeywordIndex` : BM25 keyword index backed by rank_bm25. Usage: index = BM25KeywordIndex.build(chunks) results = index.search("ألم في الصدر", k=5)
    - **Method** `__init__` : No description available.
    - **Method** `build` : Build BM25 index from a list of metadata dicts. Each dict must contain at least a "text" key.
    - **Method** `save` : No description available.
    - **Method** `load` : No description available.
    - **Method** `search` : Search the BM25 index. Returns: List of (metadata_dict, bm25_score) tuples, sorted descending.
- **Function** `get_bm25_index` : Return the global BM25 index (loaded once from disk).

---

src\medical_chatbot\rag\vector_store.py

Description: vector_store.py ——————————————————————————————— FAISS-based vector store for dense retrieval.

Uses IndexFlatIP (inner product) with L2-normalized vectors, which is equivalent to cosine similarity.

Functions & Classes:

- **Class** `FAISSVectorStore` : Manages a FAISS flat inner-product index. Attributes: index : The FAISS index object. metadata : List of dicts storing per-vector metadata.
    - **Method** `__init__` : No description available.
    - **Method** `build` : Build the FAISS index from pre-computed embeddings. Args: vectors : (N, D) float32 L2-normalized embeddings. metadata : List of N dicts (one per vector).
    - **Method** `save` : Save the FAISS index and metadata to disk.
    - **Method** `load` : Load a pre-built FAISS index from disk.
    - **Method** `search` : Search the index for the k nearest neighbours. Args: query_vector : (1, D) or (D,) float32 L2-normalized vector. k : Number of results to return. Returns: List of (metadata_dict, similarity_score) tuples, sorted by score descending.
- **Function** `get_vector_store` : Return the global FAISS vector store (loaded once from disk).

---

src\medical_chatbot\rag_*init*_.py

Functions & Classes:

- No main functions or classes defined.

---

src\medical_chatbot\tools\citation_tool.py

Description: citation_tool.py ——————————————————————————————— Formats retrieved chunks into numbered citations that the Arabic Medical Response Agent can embed directly into its answer.

Functions & Classes:

- **Class** `CitationInput` : No description available.
- **Class** `CitationGroundingTool` : No description available.
  - **Method** `_run` : No description available.

---

src\medical_chatbot\tools\classifier_tool.py

Description: classifier_tool.py ——————————————————————————————— Classifies a medical query into a category and detects emergency symptoms for priority routing.

Classification approach:

1. Emergency keyword check (always runs first)
2. Category keyword matching over Arabic + English terms
3. Fallback to "general_health"

Functions & Classes:

- **Function** `_normalize` : No description available.
- **Class** `ClassifierInput` : No description available.
- **Class** `MedicalClassifierTool` : No description available.
  - **Method** `_run` : No description available.

---

src\medical_chatbot\tools\hallucination_checker_tool.py

Description: hallucination_checker_tool.py ——————————————————————————————— Lightweight hallucination detection.

Strategy (no LLM re-call → fast inference):

1. Split generated answer into sentences.
2. For each sentence, check if any content word appears in the retrieved context (substring overlap).
3. Flag sentences with zero context support.
4. Return lightweight JSON verdict.

Functions & Classes:

- **Function** `_clean` : Remove diacritics and punctuation for matching.
- **Function** `_split_sentences` : Split on Arabic/Latin sentence terminators.
- **Class** `HallucinationCheckerInput` : No description available.

- **Class** `HallucinationCheckerTool` : No description available.
  - **Method** `_run` : No description available.

---

src\medical_chatbot\tools\hybrid_search_tool.py

Description: hybrid_search_tool.py —— Hybrid retrieval tool: Vector (FAISS) + Keyword (BM25)

Optimized fast-inference flow:

1. Run FAISS vector search
2. If top similarity < VECTOR_THRESHOLD → also run BM25
3. Merge + deduplicate results
4. Return top-K chunks as JSON

Functions & Classes:

- **Class** `HybridSearchInput` : No description available.
- **Class** `HybridSearchTool` : No description available.
  - **Method** `_run` : No description available.

---

src\medical_chatbot\tools\language_detection_tool.py

Description: language_detection_tool.py —— Detects the language of the user query using langdetect. Always signals that the response must be in Arabic.

Functions & Classes:

- **Class** `LanguageDetectionInput` : No description available.
- **Class** `LanguageDetectionTool` : No description available.
  - **Method** `_run` : No description available.

---

src\medical_chatbot\tools_*init*_.py

Functions & Classes:

- No main functions or classes defined.

---

telegram_bot\bot.py

Description: Arabic Medical Chatbot — Telegram Bot Interface Runs the same crew.run() pipeline as the Streamlit app.

Run: python telegram_bot/bot.py

Functions & Classes:

- **Function** `get_crew` : No description available.
- **Function** `state` : No description available.
- **Function** `clean_for_telegram` : No description available.

- **Function** `split_refs` : Return (answer_body, refs_text). refs_text may be empty.
- **Function** `main` : No description available.
